EXAM-READY NOTES · IA2

Software Engineering

7-Mark Answers

Module IV: Software Design

Module V: Software Testing

Module VI: SCM, Quality Assurance and Maintenance

How to use these notes

- Each answer is sized for 7 marks and follows the reference book’s terms and sections.
- Diagrams are redrawn and labelled; reproduce the simple ones in the exam.
- Amber REMEMBER boxes hold a mnemonic for quick revision.
- Green EXAMPLE boxes give a short real-life example you can add to score full marks.
- Numerical answers (Q17, Q18, Q19) show every step, formula and final result.

Contents

Module IV: Software Design 3

Q5. Explain Cohesion and Coupling. Explain different types of cohesion. 3

Q8. Explain Coupling. Explain different types with detailed examples. 4

Q20. What are the qualities of good design? What is modularity? 5

Q21. What is data-centered architecture and design principles? 6

Module V: Software Testing 7

Q1. List different software testing strategies. 7

Q2. Compare White box testing and Black box testing. 9

Q7. Explain the software testing process. 10

Q9. Explain the principles of software testing. 11

Q11. Difference between Alpha and Beta testing. 12

Q14. Explain the different techniques in white box testing. 13

Q16. What is Software Testing? Explain different types of software testing. 14

Q17. Analyse the code and perform Basis Path testing. 16

Q18. (a) Analyse the code and find the Cyclomatic complexity and write the independent paths. 17

Q19. Boundary Value Analysis and Equivalence Class Partitioning: “An employee salary field accepts values from ₹10,000 to ₹1,00,000.” 18

Module VI: SCM, Quality Assurance and Maintenance 20

Q3. Define Risk. What are the different categories of risks? 20

Q4. Explain the RMMM plan with a suitable example. 21

Q6. Explain SQA in detail. 22

Q10. Explain software Re-engineering. 23

Q12. What is SCM? Explain Version and Change Control. 24

Q13. Explain Risk and its types. Explain the setting up or generating of the RMMM plan. 27

Q15. Compare FTR and Walkthrough. 28

Module IV: Software Design

Questions 5, 8, 20, 21

Q5 Explain Cohesion and Coupling. Explain different types of cohesion. — 7 Marks

Reference book: Sec. 4.4.2, 4.5, 4.6

A module should do one well-defined job and depend as little as possible on other modules. This is called **functional independence**, and it is the result of modularity, abstraction and information hiding. It is measured by two criteria: **cohesion** (how strongly the elements inside a module belong together) and **coupling** (how much modules depend on each other). A good design has **high cohesion and low coupling**: tight inside, loose outside.



Fig. 1: High cohesion inside each module, a single thin link (low coupling) between them

Cohesion

Cohesion is the degree to which the components of a module are related to one another. Types, from worst to best:

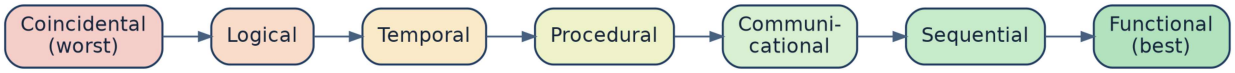


Fig. 2: Spectrum of cohesion (red = poor, green = good)

Type	Meaning	Example
Coincidental	Elements are grouped arbitrarily, with no real relation	A utility module holding printReport(), calcTax() and openFile()
Logical	Elements do similar kinds of operations, and a flag selects one	One module that reads input from keyboard, file or disk depending on a flag
Temporal	Functions are performed within the same time span; typical of start-up and shut-down	initialise(): open files, reset counters, load settings
Procedural	Elements are related only by the order of execution; a loop is a typical case	Check permission, then open file, then read record
Communicational	Different functions, but all use the same input or produce the same output	Update, print and audit the same customer record
Sequential	The output of one function is the input of the next	Read marks, validate marks, compute grade
Functional	All elements contribute to a single function	computeGST()

Advantages of high cohesion: better program design, easy reuse, more reliable, easy to understand and maintain. Low cohesion gives the opposite: hard to maintain, cannot be reused, hard to understand, less reliable.

Coupling

Coupling is the degree of dependence between modules. Highly coupled modules force changes in each other; loosely coupled modules are almost independent. The types are data, stamp, control, common, content, external and message coupling (see Q8 for details).

Point	Cohesion	Coupling
Concerns	Inside one module	Between modules
Desired level	High	Low
Benefit	Readability, reusability, reliability	No ripple effect, faster development, easy testing

REMEMBER

Cohesion types from worst to best: **'Cats Love To Play, Cats Seldom Fight'** (Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional). Picture a junk drawer (coincidental) versus a cutlery drawer (functional).

Q8 Explain Coupling. Explain different types with detailed examples. — 7 Marks

Reference book: Sec. 4.6, 4.6.1 to 4.6.3

Coupling is the measure of **interdependence between two modules**. If modules are highly coupled, a change in one forces changes in the others; if they are loosely coupled, they are almost independent. Good design always aims for low coupling.



Fig. 1: Spectrum of coupling, from worst (content) to best (message)

Type	Meaning	Example
Data	Modules pass data through an argument list and every element is used	area(length, width)
Stamp	A whole data structure is passed but only some of its elements are used	Pass the full Student record to a function that needs only the name
Control	The data passed controls the internal logic of the other module (a control flag)	print(doc, mode), where mode selects the branch inside print()
Common	Modules communicate through a global area; hard to track when global data changes	Two modules reading and writing the global variable totalMarks
Content	One module refers to the internals of another; the worst type and must not be used	Module A changes the local data of module B or jumps into the middle of B
External	Modules share an externally imposed data format, protocol or device interface	Two modules depending on the same file layout or the same hardware port
Message	Modules communicate only by passing messages, so they stay independent	An Order object sends notify() to a Customer object

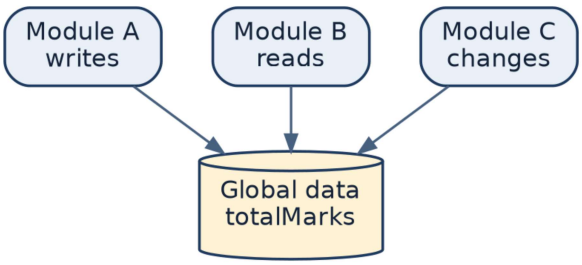


Fig. 2: Common coupling. A change made by module C can silently break A and B, and the fault is hard to trace.

Advantages of low coupling: no ripple effect, components can be reused, the system can be built faster, and testing is simpler. **Disadvantages of high coupling:** a change in one component forces changes in others, and development slows down.

How to reduce coupling: pass only the data that is needed, use well-defined interfaces, hide internal details, avoid global variables and avoid control flags.

> **REMEMBER**
> Worst to best: **'Cats Chase Every Cow; Sheep Don't Move'** (Content, Common, External, Control, Stamp, Data, Message). A wall socket is low coupling: any standard plug fits and nothing else is shared. Soldering wires into each other's insides is content coupling.

Q20 What are the qualities of good design? What is modularity? 7 Marks

Reference book: Sec. 4.1.1, 4.4

Mitch Kapor said good software design must show three things: **Firmness** (no bugs that stop it working), **Commodity** (suitable for the purpose it was built for) and **Delight** (a pleasant experience to use).

Qualities of a good design

- **Innovative:** a completely new design or a redesign that improves an existing product.
- **Functional:** fulfils every intended function and solves the user's problem.
- **Honest:** shows the functions and values it really offers and never promises what it cannot deliver.
- **User-oriented:** built around the user's needs, giving the user satisfaction.
- **Correct:** implements all the functionality required by the SRS.
- **Modular, with a clear architecture:** logically divided into elements or subsystems, built from the analysis model, and open to change.

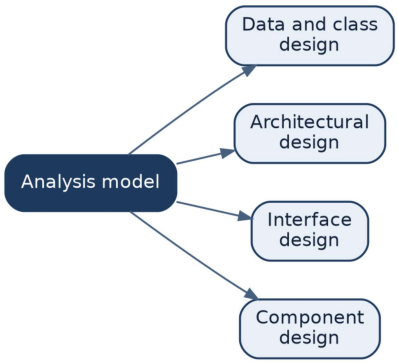


Fig. 1: The design is derived from the analysis model

Modularity

Modularity means dividing the software into **separately named and addressable components called modules**, so that each module can be developed independently and then integrated. It is divide and conquer: the way a car is built from engine, wheels and brakes.

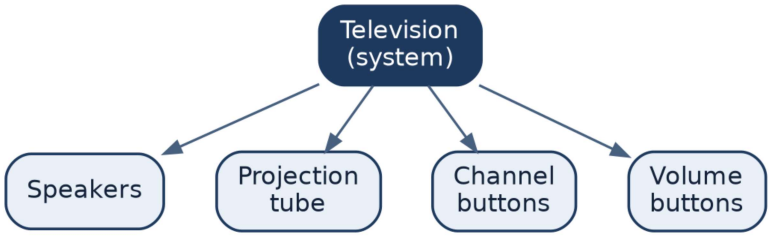


Fig. 2: A television is made of modules; each has its own function, and together they give the full functionality

Advantages of modularity: smaller components are easy to maintain; the program can be divided by function; levels of abstraction can be applied; parts can be developed in parallel and executed concurrently; high-cohesion modules can be reused; security can be achieved. Overall it reduces complexity and makes change easier.

REMEMBER
Kapor = **F-C-D**. Qualities = **'I'm Fully Honest, Utterly Correct'** (Innovative, Functional, Honest, User-oriented, Correct).

Q21 What is data-centered architecture and design principles? — 7 Marks

Reference book: Sec. 4.2, 4.8.3

Data-centered architecture has two components: a **central data structure (data store)**, such as a database or file, which holds the current state of the data, and a **collection of client software** that adds, deletes, updates and reads that data. Clients work independently of one another and talk only to the data store.

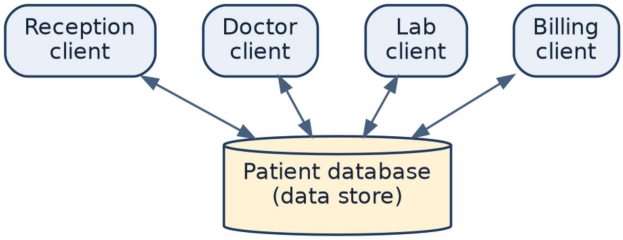


Fig. 1: Data-centered architecture in a hospital system

In a hospital system, the patient database is the data store; the reception, doctor, lab and billing programs are clients. In the **blackboard variation**, the data store informs the clients when data of interest changes, for example when a lab result arrives.

Advantages: the data store is independent of the clients; clients work independently; new clients are easy to add; modification is easy; data integration is achieved. **Disadvantages:** the data store is a single point of failure and a possible bottleneck.

Other architectural styles	One-liner
Data-flow	Data passes through a chain of filters
Call and return	A main program calls subprograms, including remote procedure calls
Object-oriented	Cooperating objects pass messages
Layered	Each layer serves the layer above it

Design principles

1. The design process should not suffer from **tunnel vision**.
2. The design should be **traceable** to the analysis model.
3. The design should **not reinvent the wheel**.
4. It should **minimise the intellectual distance** between the software and the real-world problem.
5. It should exhibit **uniformity and integration**.
6. It should be structured to **accommodate change**.
7. It should **degrade gently** when unusual data, events or conditions occur.
8. Design is not coding, and coding is not design.
9. It should be **assessed for quality while it is being created**, not afterwards.
10. It should be **reviewed** to minimise conceptual errors.

REMEMBER
'Two Teachers Reuse Interesting Unit Charts During Design And Review' gives the first letters of the ten principles in order.

Module V: Software Testing

Questions 1, 2, 7, 9, 11, 14, 16, 17, 18, 19

Q1 List different software testing strategies. 7 Marks

Reference book: Sec. 6.2 to 6.6

Testing follows the software process like a spiral: it starts at the centre with single units and moves outward through integration and validation to the whole system.

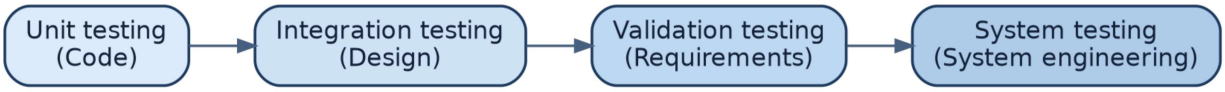


Fig. 1: Testing strategy as a spiral, small to big

REMEMBER
U-I-V-S, small to big.

1. **Verification and validation.** Verification asks ‘are we building the product right?’ and uses reviews, inspections and walkthroughs. Validation asks ‘are we building the right product?’ and uses actual testing.
2. **Unit testing.** Tests the smallest testable part (a function or method) to confirm it works as designed. It is white box testing, done by developers before integration. It finds errors in local data structures, boundary conditions, independent paths, error handling, initialisation and arithmetic.
3. **Integration testing.** Described in detail below.
4. **Validation testing.** Checks the software against the requirements set during analysis (functional, behavioural and performance), and includes alpha and beta testing.
5. **System testing.** Tests the complete integrated product together with hardware, people and database, through recovery, security, stress and performance tests.
6. **Regression testing.** Re-runs old test cases after a change to confirm that existing functions still work.

Integration testing in detail

All units are combined and tested as a group, always after unit testing. The focus is the **data communication between units**.

Why it is needed: each module is written by a different developer; requirements may change after unit testing; the interface between modules and the database may have errors; external hardware interfaces may fail; exceptions may not be handled properly; and we must check that modules work together.

	Stub	Driver
Replaces	The called (lower) module that is not ready	The calling (upper) module that is not ready
Job	Returns fixed values to the module under test	Sends test inputs to the modules under test
Used in	Top-down integration	Bottom-up integration
Remember	Stub: Sub-module is missing	Driver sits on top and Drives the others

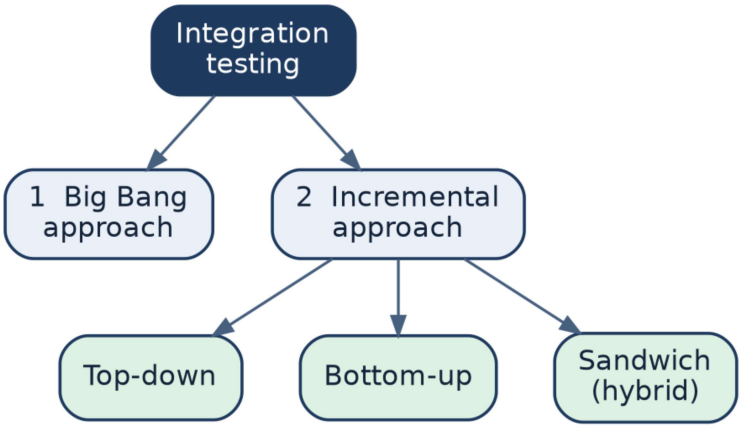


Fig. 2: Approaches to integration testing

- **Big Bang:** all modules are combined at once and the whole is tested. Advantage: suits small projects. Disadvantages: finding a defect is difficult; many interfaces must be checked and some may stay untested; testing starts late, so little time is left.
- **Incremental:** logically related modules are combined and tested, then more modules are added one by one until the whole system is covered. Stubs and drivers are used.
- **Top-down:** start with the main module and move down, using stubs for the lower modules, replacing each stub by the real module in turn. Major design flaws are found early, but many stubs are needed.
- **Bottom-up:** start with the lowest modules, using a driver for the upper module, and move upward. No stubs are needed and low-level modules are tested thoroughly, but the complete system exists only at the end.
- **Sandwich (hybrid):** top-down and bottom-up run together and meet in the middle layer. It combines the strengths of both but is more complex and costly.

Approach	Starts at	Needs	Best for
Big Bang	All modules together	Nothing extra	Small projects
Top-down	Main module	Stubs	Finding design flaws early
Bottom-up	Lowest modules	Drivers	Thorough low-level testing
Sandwich	Top and bottom together	Both	Large systems

Characteristics of a good testing strategy: technical reviews first; testing proceeds from components outward; different techniques suit different stages; an independent test group for large projects; debugging is accommodated; both low-level and high-level tests are included; progress is measurable.

Q2 Compare White box testing and Black box testing. 7 Marks

Reference book: Sec. 6.8, 6.11, 6.11.4

White box testing (clear or glass box) tests the internal structure and logic of the code.

Black box testing (behavioural or specification-based) tests the functionality using inputs and outputs, without knowing the internal structure. Its techniques are boundary value analysis, equivalence partitioning, cause-effect graphing and error guessing.

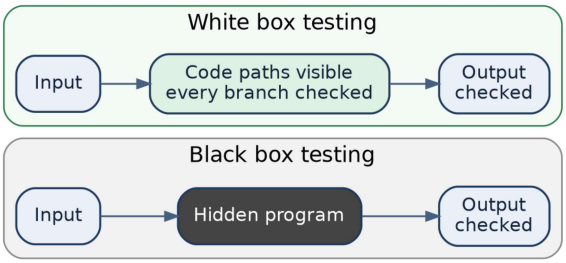


Fig. 1: Black box checks the output only; white box looks inside the code

Parameter	Black box testing	White box testing
Basis	Requirement specification	Detailed design and source code
Knowledge needed	No programming knowledge	Programming and implementation knowledge
Level applied	Higher levels: system and acceptance	Lower levels: unit and integration
Performed by	Software testers	Developers
Aim	What the system does	How the system does it
Focus	Functionality	Code, security and flow of control
Cost	Less expensive	More expensive
Techniques	BVA, equivalence partitioning, cause-effect, error guessing	Basis path, condition, data flow, loop testing
Communication between modules	Can be verified	Cannot be verified

EXAMPLE

Login screen. Black box: enter a correct and a wrong password and check the messages. White box: read the validate() code and make sure every if/else branch and loop runs at least once. A gray box test mixes the two: the tester knows a little about the internals but tests from outside.

REMEMBER

White = Wiring (you see the code inside). **Black = Behaviour** (you see only what comes out).

Q7 Explain the software testing process. 7 Marks

Reference book: Sec. 6.1.6

The testing process is a sequence of activities that certify the software system. Every activity has **entry criteria** (conditions that must be met before it begins) and **exit criteria** (conditions that must be met before it ends).

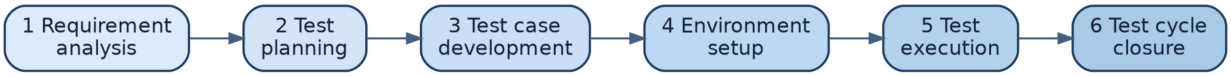


Fig. 1: Phases of the software testing process

Phase	What happens	Entry criteria	Exit criteria / output
Requirement analysis	Functional and non-functional requirements are studied to decide whether they are testable	Requirement document available	Agreed list of testable requirements
Test planning	Test strategy, effort, cost and resources are estimated by the test manager	Requirements analysed	Approved test plan and effort estimate
Test case development	Test data, test cases and scripts are created, reviewed and reworked	Test plan ready	Reviewed test cases and scripts
Environment set-up	Hardware, software and network are prepared; a smoke test checks the set-up	Plan and cases ready	Smoke test passes
Test execution	The test team runs the cases, reports defects to developers and retests after fixing	Environment and cases ready	Defect logs, all planned cases run
Test cycle closure	Completion criteria (time, cost, business objectives) are checked and a closure report with metrics is prepared	Execution complete	Test closure report

EXAMPLE

Login module. Requirement: a user can log in with a valid ID. Plan: 2 testers, 3 days. Cases: valid ID, wrong password, blank fields. Execution: run the cases and log a defect if a wrong password still logs in. Closure: report that 19 of 20 cases passed and the one defect was fixed and retested.

Testing finds a defect; debugging finds its cause and fixes it. They are separate activities.

REMEMBER

‘Real Programmers Create Environments, Then Close’ (Requirement, Planning, Case, Environment, Test execution, Closure).

Q9 Explain the principles of software testing. 7 Marks

Reference book: Sec. 6.1.4

There are seven principles of software testing:

1. **Complete (exhaustive) testing is not possible.** There is never enough time to test every input and module, so we test as much as possible guided by risk assessment, which means guessing where defects are most likely.
2. **Defect clustering.** About 80% of the defects are found in 20% of the modules; experience helps find these risky modules.
3. **Pesticide paradox.** If the same test cases are repeated, they stop finding new defects, so test cases must be reviewed and new ones added regularly.
4. **Testing shows the presence of defects, not their absence.** Even after testing we cannot guarantee that the system has no defects.
5. **Absence-of-error fallacy.** Software that is 99% bug-free is still useless if it was tested against the wrong requirements; it must meet the user's needs.
6. **Early testing.** Testing should begin as early as possible in the SDLC, even while requirements are gathered, because a defect found early is cheaper to fix.
7. **Testing is context dependent.** An online shopping site is tested differently from a stand-alone application.

Principle	Everyday example
Exhaustive testing impossible	A 10-digit phone field has 10 billion possible inputs
Defect clustering	The payment module holds most of the bugs of a shop app
Pesticide paradox	The same pesticide stops killing insects; the same tests stop finding bugs
Presence, not absence	100 passed tests do not prove there are no bugs
Absence-of-error fallacy	A bug-free app that nobody wants is still a failure
Early testing	A mistake in the SRS is cheap to fix; in production it is costly
Context dependent	A bank app needs security tests; a game needs performance tests

Objectives of testing: find defects introduced by the developer, give information about quality, make sure the system meets business and user requirements, and gain the customer's confidence.

> **REMEMBER**
> **'Every Dev Prays: Testing Always Ends Clean'** (Exhaustive, Defect clustering, Pesticide, Testing shows presence, Absence of error, Early, Context).

Q11 Difference between Alpha and Beta testing. — 7 Marks

Reference book: Sec. 6.5.2, 6.5.3

Alpha testing is acceptance testing done to find bugs before the product is released to end users; it is the last stage before release into the real world. It is performed by in-house engineers or QA staff in two phases: first the developers test and debug quickly, then the software quality analyst team tests in a set-up similar to the user's environment.

Beta testing is performed at the customer's location by the intended users, who judge whether the software satisfies their needs and expectations. It is also called user acceptance, customer validation or pre-release testing.



Fig. 1: Alpha comes first at the developer's site; beta follows at the customer's site

Parameter	Alpha testing	Beta testing
Performed by	In-house developers and QA staff	Real end users or customers
Place and environment	Developer's site, controlled environment	Customer's site, real-world environment
Order	First, before beta	After alpha, just before release
Purpose	Find and fix bugs not found earlier; assure quality before beta	Collect user feedback; confirm the product is ready for real use
Fixing issues	Important issues are corrected by developers without delay	Feedback is used to improve the product or the next release
Duration	Shorter, in two phases	A few weeks
Benefit	Early insight into reliability and usability; reduces time to market	Lowers failure risk, improves quality and customer satisfaction at low cost

Types of beta testing

Type	Meaning	Example
Traditional	Product given to targeted users and all-round data collected	A bank gives a new app to 50 chosen customers
Public	Released publicly online; anyone can give feedback	Open beta of a mobile game
Technical	Released inside the organisation; employees give feedback	Staff try the new HR portal
Focused	Released to get feedback on specific features	Testing only the new search bar
Post-release	Released in the market; data used for the next version	Version 1 feedback shapes version 2

REMEMBER

Alpha = Ahead of release, At home. Beta = Buyer's place.

Q14 Explain the different techniques in white box testing. 7 Marks

Reference book: Sec. 6.8 to 6.10

White box testing (clear box testing) checks the **internal working of the code**. Test cases are derived from the source code, so the tester must know the code and its logic. The steps are: understand the source code, then create test cases and execute them.

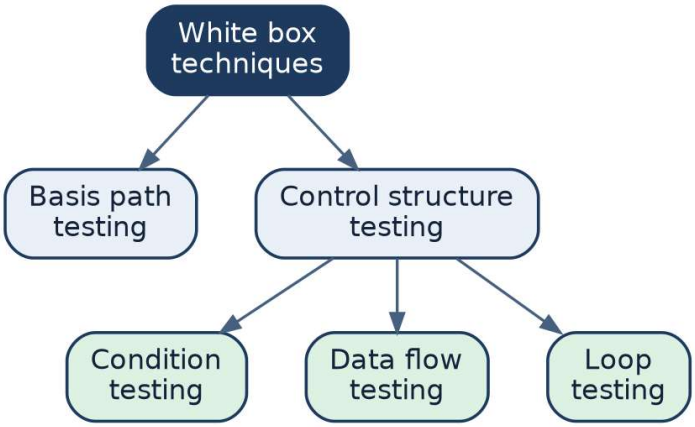


Fig. 1: White box techniques

1. Basis path testing (proposed by Tom McCabe)

It guarantees that every statement is executed at least once.

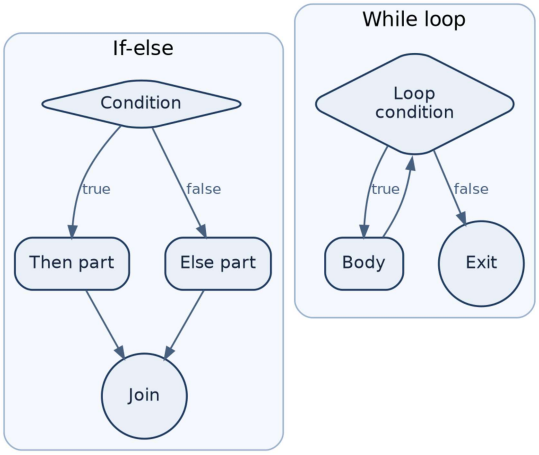


Fig. 2: Flow graph notation for if-else and while constructs

Term	Meaning
Node	A statement or block of statements
Edge	Flow of control from one node to the next
Predicate node	A node containing a condition
Region	An area bounded by edges and nodes (the outside area counts too)
Independent path	A path that introduces at least one new statement or condition

Steps: (1) draw the flow graph from the code; (2) compute cyclomatic complexity **V(G) = E − N + 2 = P + 1 = number of regions**; (3) list the independent paths (their number equals V(G)); (4) prepare one test case per path. *Example:* a graph with 13 nodes, 17 edges and 5 predicate nodes has V(G) = 17 − 13 + 2 = 6 = 5 + 1, so 6 test cases are needed.

2. Control structure testing

Condition testing. Exercises every logical condition in the program, making each simple condition both true and false. For `if (age >= 18 && hasID)` the four combinations TT, TF, FT, FF are tested.

Data flow testing. Selects test paths according to where variables are defined and used. In `x = 5;` x is defined; in `print(x);` it is used. Test paths run from each definition to each use.

Loop testing.

Loop type	How to test
Simple loop (max n passes)	Skip the loop; 1 pass; 2 passes; m passes (m < n); n−1, n and n+1 passes
Nested loops	Start with the innermost loop while the outer loops are at their minimum; work outward, keeping the others at minimum or typical values, until all loops are tested
Concatenated loops	If independent, test each as a simple loop; if the first loop's counter feeds the second, treat them as nested
Unstructured loops	Redesign them using structured constructs

Advantages: can be applied early, finds hidden errors, helps code optimisation, easy to automate.
Disadvantages: complex and expensive, impractical for very large applications, and may miss missing functionality.

> **REMEMBER**
> **'Big Cats Do Leap'** (Basis path, Condition, Data flow, Loop).

Q16 What is Software Testing? Explain different types of software testing. — 7 Marks

Reference book: Sec. 6.1 to 6.6

Software testing is the procedure of **verifying whether the actual results are the same as the expected results**. It assures that the software has no defects and meets the user's requirements. A **test case** is a set of actions used to check one feature, a collection of test cases is a **test suite**, and **test data** is the input given to the modules (positive data checks expected behaviour, negative data checks handling of unexpected input).

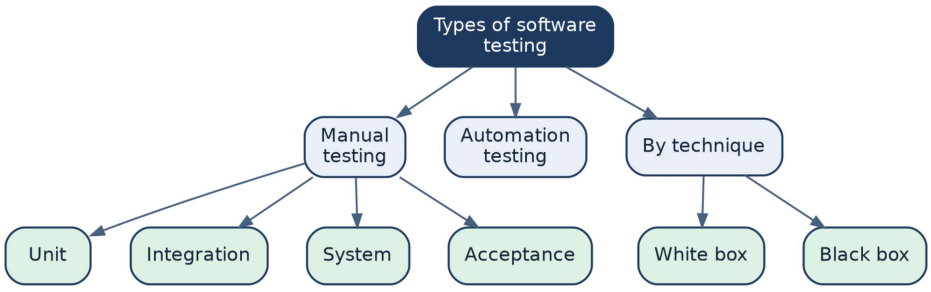


Fig. 1: Types of software testing

Type	Meaning
Manual testing	The tester acts as the end user and runs test cases without tools; always done first for a new application
Automation testing	Tools such as Selenium or QTP run recorded test suites; suits stable requirements and repeated runs
Unit testing	The smallest testable part is tested by developers using white box methods, with stubs and drivers
Integration testing	Combined units are tested for data communication: big bang, top-down, bottom-up, sandwich

Type	Meaning
System testing	The complete integrated system is tested against the requirements
Acceptance testing	Alpha testing at the developer's site and beta testing at the customer's site
White box / Black box	Code-based versus specification-based testing
Functional / Non-functional	Each function against the requirements / performance, usability, scalability and security
Regression testing	Old tests are re-run to check that a change did not break existing functions
Load testing	Finds how many users can use the system at the same time
Recovery testing	Checks how the system recovers after a crash such as a power failure
Migration testing	Checks that software moves from the old infrastructure to the new one without problems

	Manual testing	Automation testing
Resources	Needs skilled people, more time and effort	Needs tools and cost, but runs recorded suites easily
Best for	Good for ad-hoc and exploratory testing	Good for repetitive regression testing

The V-model

The V-model pairs each design stage with a test level: requirements with acceptance testing, system design with system testing, architecture design with integration testing, and module design with unit testing.

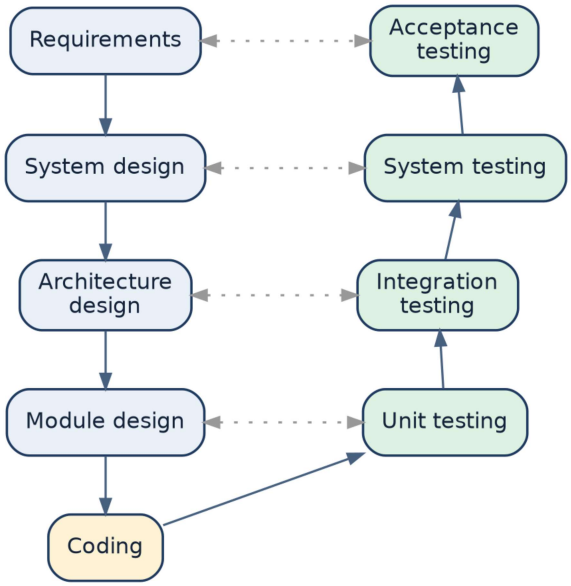


Fig. 2: V-model (dotted lines pair each stage with its test level)

REMEMBER

The four levels: **'Unit Is Super Awesome'** (Unit, Integration, System, Acceptance).

Q17 Analyse the code and perform Basis Path testing. — 7 Marks

Reference book: Sec. 6.9 (flow graph, V(G), deriving test cases)

```
1: void checkDiscount(int amount) {
2:   if (amount > 2000) {
3:     print("Apply 25% Discount");
4:   } else {
5:     print("No Discount");
6:   }
7:   print("Transaction complete");
8: }
```

Step 1: Draw the flow graph

Node	Statements	Meaning
N1	1	Entry
N2	2	Decision (predicate node)
N3	3	Apply 25% Discount
N4	5	No Discount
N5	7, 8	Transaction complete, exit

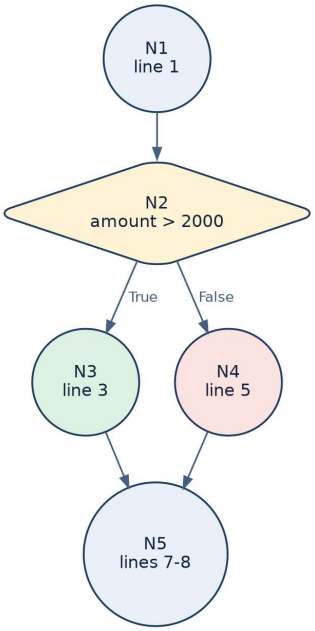


Fig. 1: Flow graph of checkDiscount()

Edges: N1→N2, N2→N3, N2→N4, N3→N5, N4→N5. So **E = 5, N = 5** and the number of predicate nodes **P = 1**.

Step 2: Cyclomatic complexity

CALCULATION

V(G) = E − N + 2 = 5 − 5 + 2 = **2**

V(G) = P + 1 = 1 + 1 = **2**

V(G) = number of regions = **2**

There is only one decision, so there are two independent paths.

Step 3: List the independent paths

- **Path 1:** N1 → N2 → N3 → N5 (lines 1-2-3-7-8)
- **Path 2:** N1 → N2 → N4 → N5 (lines 1-2-5-7-8)

Step 4: Design the test cases

Path	Input	Expected output
1	amount = 2500	Apply 25% Discount, Transaction complete
2	amount = 1500	No Discount, Transaction complete

Together the two paths run lines 1 to 8, giving 100% statement coverage and 100% branch coverage. The boundary value amount = 2000 follows the false branch (No Discount).

REMEMBER
D-C-L-T (Draw the graph, Count V(G), List the paths, Test cases), and V(G) = decisions + 1.

Q18 (a) Analyse the code and find the Cyclomatic complexity and write the independent paths. — 7 Marks

Reference book: Sec. 6.9 (flow graph, V(G), independent paths)

```
1: void swapIfGreater(int a, int b) {
2:   if (a > b) {
3:     int temp = a;
4:     a = b;
5:     b = temp;
6:   }
7:   print("Final order: " + a + ", " + b);
8: }
```

Flow graph

Statements 3, 4 and 5 are consecutive, so they form one node.

Node	Statements
N1	1 (entry)
N2	2 (decision: a > b)
N3	3, 4, 5 (swap)
N4	7, 8 (print, exit)

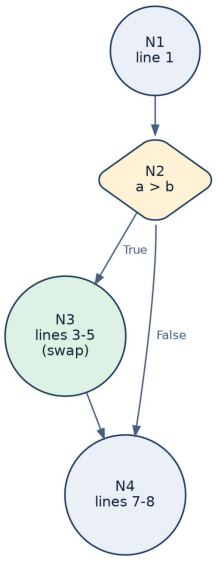


Fig. 1: Flow graph of swapIfGreater()

Edges: N1→N2, N2→N3, N2→N4, N3→N4, so **E = 4, N = 4, P = 1**.

CALCULATION
V(G) = E − N + 2 = 4 − 4 + 2 = **2**
V(G) = P + 1 = **2**, and the number of regions is **2**

Independent paths

- **Path 1:** N1 → N2 → N3 → N4 (lines 1-2-3-4-5-7-8, the swap happens)
- **Path 2:** N1 → N2 → N4 (lines 1-2-7-8, no swap)

Path	Test input	Expected output
1	a = 10, b = 4	Final order: 4, 10
2	a = 3, b = 8	Final order: 3, 8

If a = b, the condition is false and path 2 runs. A complexity of 2 means a simple module that is easy to test. Both paths together cover every statement and every branch.

Q19 Boundary Value Analysis and Equivalence Class Partitioning: “An employee salary field accepts values from ₹10,000 to ₹1,00,000.” 7 Marks

Reference book: Sec. 6.11 (BVA and equivalence partitioning, brief); worked method is standard

Step 1: Equivalence class partitioning

Split the inputs into families and test one value from each.

Class	Range	Type	Test value	Expected
EC1	Below 10,000 (up to 9,999)	Invalid	5,000	Rejected
EC2	10,000 to 1,00,000	Valid	50,000	Accepted
EC3	Above 1,00,000 (1,00,001 onwards)	Invalid	1,50,000	Rejected

Extra invalid inputs worth testing: non-numeric text (abc), a blank field, a negative value.

Step 2: Boundary value analysis

Test the edges and the values next to them: minimum − 1, minimum, minimum + 1, a nominal value, maximum − 1, maximum and maximum + 1. Here minimum = 10,000 and maximum = 1,00,000.

Test	Value	Boundary point	Expected
1	9,999	Just below minimum	Rejected
2	10,000	Minimum	Accepted
3	10,001	Just above minimum	Accepted
4	50,000	Nominal	Accepted
5	99,999	Just below maximum	Accepted
6	1,00,000	Maximum	Accepted
7	1,00,001	Just above maximum	Rejected

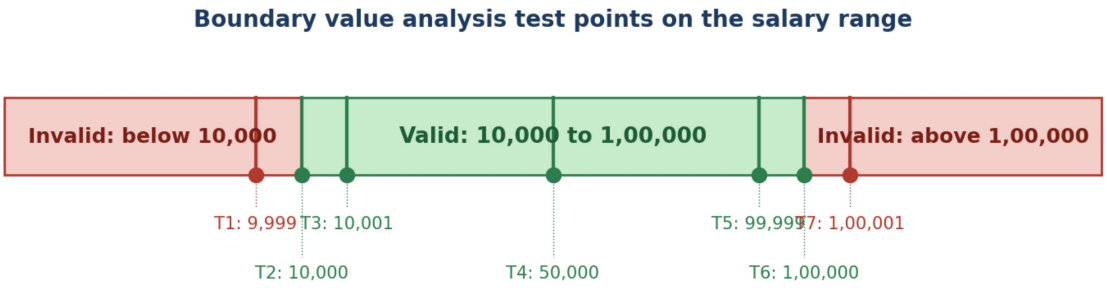


Fig. 1: Equivalence classes and the seven boundary test points

Result: 3 equivalence-class tests plus 7 boundary tests. Boundary tests 1, 2, 6 and 7 are the most important, because mistakes such as using > instead of >= appear exactly at the edges.

	Equivalence partitioning	Boundary value analysis
Idea	One value per family	Values at and around the edges
Order	Done first	Done after partitioning
Catches	Wrong handling of whole ranges	Off-by-one mistakes

REMEMBER

EP = ‘families’; BVA = ‘edges and neighbours’ (min−1, min, min+1, nominal, max−1, max, max+1).

Module VI: SCM, Quality Assurance and Maintenance

Questions 3, 4, 6, 10, 12, 13, 15

Q3 Define Risk. What are the different categories of risks? — 7 Marks

Reference book: Sec. 5.1 to 5.4

Risk is the possibility of suffering loss. In a project the loss shows up as lower quality of the product, higher cost, delayed completion or complete failure.



Fig. 1: Categories of software risk

Category	Meaning	Typical causes
Schedule / time-related	Running behind schedule; affects delivery directly	Wrong time estimates, poor resource allocation, idle resources, poor understanding of project complexity, unexpected growth of scope
Budget / financial	Monetary risk of overrunning the budget	Wrong budget estimate, cost overruns from idle resources, scope expansion, poor tracking of finances
Operational / procedural	Risks in day-to-day project activities	Lack of training, poor team communication, no escalation or conflict-resolution process, unclear priorities
Technical / functional / performance	Risks in the product itself	Insufficient testing, trade-off between maximum functionality and peak performance, changing requirements (freeze a cut-off date for requirements)
Unavoidable	Cannot be foreseen or planned for	Change in government policy, obsolescence due to a rival's new technology, loss of contracts at the customer's end

EXAMPLE

College website project. The client adds features late (schedule and budget risk); the team was never trained on the tool (operational); the site slows down for 5,000 users (technical); a new government rule changes the admission form (unavoidable).

Risk management reduces such losses: **risk assessment** (identification, analysis, prioritisation) followed by **risk containment**. Impact is rated 1 (catastrophic), 2 (critical), 3 (marginal) or 4 (negligible), and risk exposure is **RE = P × C** (probability × cost); for example 0.7 × ₹1,00,000 = ₹70,000.

REMEMBER

'Students Borrow Old Textbooks Usually' (Schedule, Budget, Operational, Technical, Unavoidable).

Q4 Explain the RMMM plan with a suitable example. 7 Marks

Reference book: Sec. 5.5, 5.5.1

RMMM stands for **Risk Mitigation, Monitoring and Management**. It is the strategy that guides the team in dealing with risk. If the team is proactive, avoiding the risk (mitigation) is always the best strategy.

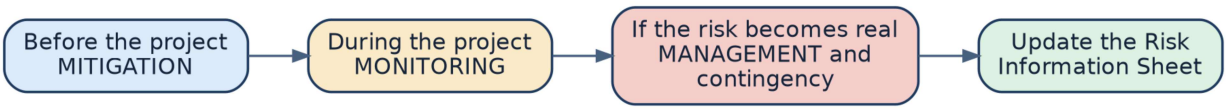


Fig. 1: The three parts of the RMMM plan

- **Mitigation:** steps taken in advance to avoid the risk.
- **Monitoring:** tracking the factors that show whether the risk is becoming more or less likely, and checking that the mitigation steps are working. It also collects data for future risk analysis.
- **Management and contingency planning:** what to do once the risk has actually occurred.

Supporting step: the risk table

List every risk with its category, probability, impact (1 to 4) and a pointer to its RMMM entry; sort by probability and impact and draw a cut-off line below which risks get no further attention.

Risk	Category	Probability	Impact	RMMM
Staff turnover	Project	70%	2 (critical)	RMMM-1
Requirement changes	Project	50%	3 (marginal)	RMMM-2
Server crash in demo	Technical	20%	2 (critical)	RMMM-3

Example: risk r1, high staff turnover

From past history the likelihood is 0.70 and the impact is critical.

Part	Actions
Mitigation (before the project starts)	Meet the staff to find the reasons for turnover (poor working conditions, low pay, competitive job market) and reduce them; organise teams so that information is shared widely; define documentation and work-product standards; assign backup staff for every critical person
Monitoring (during the project)	Watch team attitude and cohesion, pay and benefits, and the availability of other jobs; check that documentation is produced on time and mitigation steps are applied
Management / contingency (if people actually leave)	Use the backup staff, rely on the documentation and shared knowledge, temporarily refocus resources and bring newcomers up to speed

Mitigation itself costs extra time and money. Teams usually record each risk on a **Risk Information Sheet (RIS)** instead of one large RMMM document.

> **REMEMBER**
> The three M's are **'Avoid, Watch, Rescue'**.

Q6 Explain SQA in detail. — 7 Marks

Reference book: Sec. 5.7 to 5.11

Software Quality Assurance (SQA) is a set of activities that guarantees the quality of the software engineering **process**. It makes sure that the developed software meets and complies with the predefined or standard quality specifications. SQA is an ongoing activity throughout the SDLC, and the quality of the process directly affects the quality of the product.

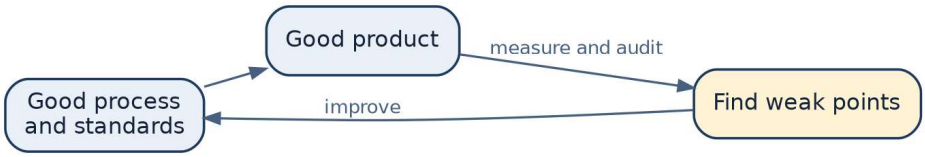


Fig. 1: The SQA improvement loop

SQA tasks: identify weaknesses in the process and correct them so that the process keeps improving. **SQA activities:** (1) process definition and implementation, (2) auditing, (3) guidance. They cover development methods, project management, configuration management, requirements management, estimation, design and testing.

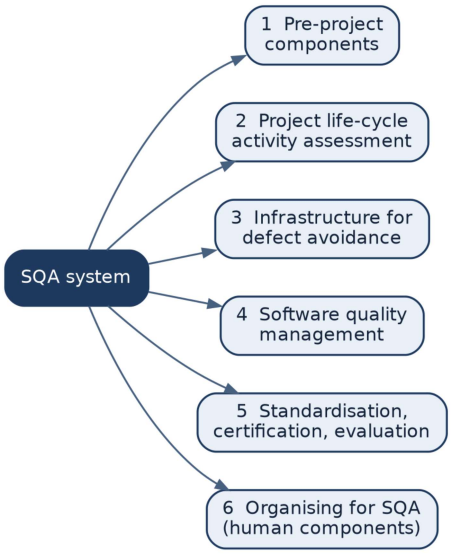


Fig. 2: The six components of an SQA system

Component	Role
Pre-project	Clear commitments on resources, schedule and budget. It includes the deal review (checking the proposal or contract) and the development and quality plans
Life-cycle assessment	Development stage (reviews, expert opinions, software testing) and operation-maintenance stage
Infrastructure	Removes or reduces errors using the organisation’s SQA experience
Quality management	Manages development and maintenance and prevents schedule and budget failures
Standardisation	Applies quality-management standards and project-process standards, and supports certification
Human components	Managers, testers, the SQA unit and committee members who start and maintain SQA procedures

Process quality management involves setting essential process standards, monitoring the development process and reporting on the software.

	Quality Assurance	Quality Control	Testing
Focus	Process: prevents defects	Product: detects defects	One way of doing quality control
When	Throughout the SDLC	After the product is built	After code exists
Example	Setting coding standards, audits	Reviewing a finished module	Running test cases

REMEMBER

The six components: **'People Love Interesting Math Stories, Honestly'** (Pre-project, Life-cycle, Infrastructure, Management, Standardisation, Human). The improvement loop is Plan, Do, Check, Act.

Q10 Explain software Re-engineering. — 7 Marks

Reference book: Sec. 6.13 to 6.15

Software re-engineering is the process of examining and restructuring existing (legacy) software, either a small module or the complete system, to improve its quality, maintainability and reusability **without changing what it does**.

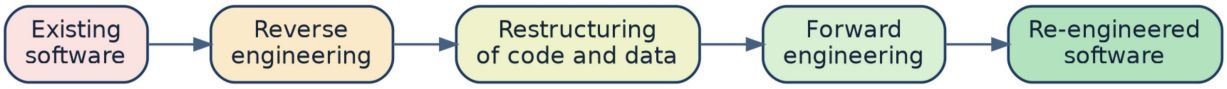


Fig. 1: The re-engineering process

Re-engineering procedure

1. Decide which components (or the complete software) should be re-engineered.
2. Perform reverse engineering to find out the features of the existing software.
3. Restructure the source code if needed, for example convert function-oriented programs into object-oriented ones.
4. Restructure the data if needed.
5. Use forward engineering to generate the re-engineered software.

Term	Meaning
Reverse engineering	Getting the system specification by analysing and understanding the existing system; it goes backwards from the maintenance phase to the requirements
Restructuring	Changing the structure of code or data to reduce complexity, without changing behaviour
Forward engineering	Normal engineering carried out after reverse engineering, using its output (the specification)

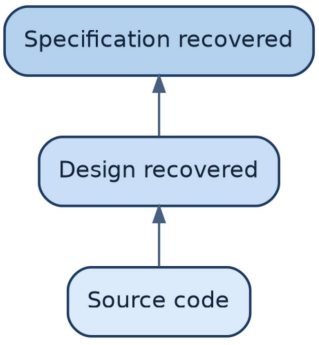


Fig. 2: Reverse engineering climbs the levels of abstraction

Reverse engineering climbs the levels of abstraction: code, then design, then specification. It is used to recover source code from machine code, to repair a bug, to improve performance and to detect unsafe content such as a virus.

EXAMPLE
An old payroll system has no documents. Reverse engineering recovers its design; restructuring cleans the code and the data layout; forward engineering rebuilds it in a modern language with a better interface, while the payroll rules stay the same.

Why it is needed: maintenance is a large part of the total cost, old code is hard to understand, and undocumented changes cause trouble later. **Benefits:** easier maintenance, lower long-term cost, better documentation, reusable parts. **Drawbacks:** it takes time and money, and old logic may be hard to recover.

Types of software maintenance (closely related, and in your syllabus)

Type	Meaning	Example	Share of effort
Corrective	Fix defects found by users or testers, including emergency patches	Fix a crash	20%
Adaptive	Adjust to a changed environment, rule or policy	Accept a new currency	25%
Perfective	Add features or improve performance	Add a new report	50% (largest)
Preventive	Prevent future problems: documentation, optimisation, restructuring	Clean up complex code	5%

REMEMBER
Re-engineering = **'Rewind, Repair, Rebuild'** (Reverse, Restructure, Forward). Maintenance types = **'Cats Always Purr Properly'** (Corrective, Adaptive, Perfective, Preventive).

Q12 What is SCM? Explain Version and Change Control. 7 Marks

Reference book: Sec. 5.6, 5.6.1 to 5.6.4

Software Configuration Management (SCM) is a system for managing the evolution of software products, both during development and during all stages of maintenance. It identifies the items in a system, controls their changes and release, records their status, and verifies that they are complete and correct. A software product here means the programs, procedures, documentation and data, plus the supporting tools.

WHY IT IS NEEDED
Think of version history in a shared document: many people, many versions, one item that must stay traceable. Without SCM the wrong version gets shipped, two developers overwrite each other's work, and nobody knows who changed what.

Benefits: a snapshot of the changing software; tracking of concurrent development; no two developers changing the same module at once; visibility of the whole project to every developer; faster progress; every phase organised and documented.

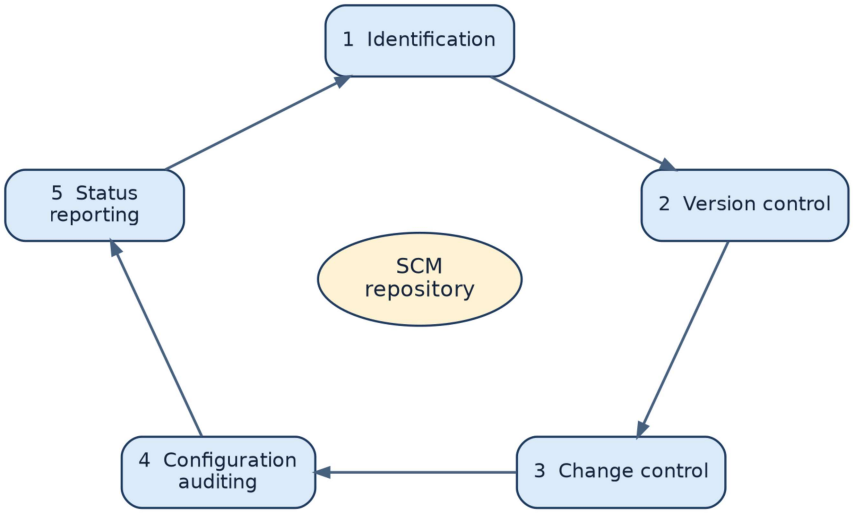


Fig. 1: SCM process, five tasks in a cycle

The SCM process has five tasks, run as a cycle: every approved change goes round the loop again. Its objectives are to identify all items of the configuration, manage changes to them, ease the building of different versions, and keep quality consistent as the configuration evolves.

> **REMEMBER**
> **'I Voted, Changed, Audited, Reported'** gives I-V-C-A-R: Identification, Version control, Change control, Auditing, Reporting.

1. Configuration identification

What: decide which items will be managed and give each a unique name and an owner.

1. Select the Configuration Items (CIs).
2. Record their functional and physical characteristics.
3. Name each using a naming convention, a label plus a version number such as SRS_v1.2.
4. Assign the person or authority responsible.
5. Arrange them in a hierarchy of configuration items, components and units.

Items include: management plans, requirement and design specifications, source and executable code, libraries, databases, test plans and data, user and maintenance documents, and the support software used in development. *Output: a list of CIs with names, versions and owners.*

A **baseline** is a specification or product that has been formally reviewed and agreed upon, serves as the basis for further development, and can be changed only through formal change control. It is a snapshot of the software at a point in time.

Baseline	Set when	Remember as
Functional	The software specification is approved	What it must do
Allocated	The software requirements specification is approved	Who does what
Developmental	The design is approved	How it is built
Product	The last Functional Configuration Audit is completed	Ready to ship

2. Version control

Version control is a combination of procedures and tools used to manage the **different versions of configuration objects** created during the software process.

1. Keep all CIs in the repository (project database).
2. A version management system stores every version of each object.

3. Developers check out a copy, edit it and check in a new version.
4. A make facility builds a particular version of the system.
5. Change sets, named sets of all the changes, are applied to the baseline to build a specific version.
6. Issue and bug tracking records the status of each object.

EXAMPLE
Version numbers 1.0, then 1.1 for a small fix and 2.0 for a major release. Git is a modern tool of this kind.
Output: every version can be rebuilt on demand.

The **repository** is a database that is the centre for storing software engineering information. It provides data integrity, sharing and integration, and acts as a hub for tools. Earlier, one person had to memorise everything, which did not work.

3. Change control

Changes occur in every phase, from changed requirements to defects found in testing, and they must be made to the right version. Change control makes sure that a change is requested, judged, approved, built, tested and released in a controlled way.

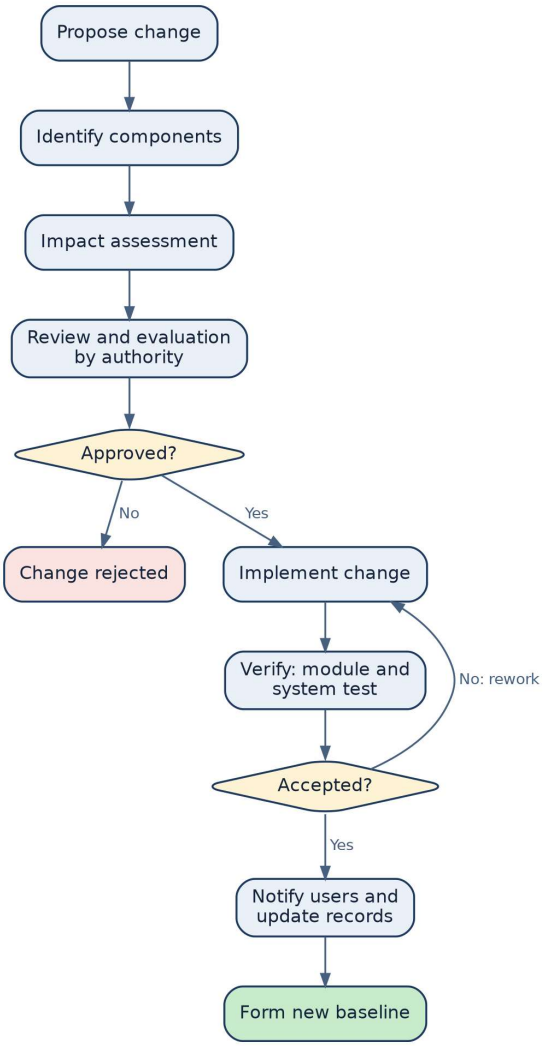


Fig. 2: The change control process

1. Propose the change (change request).
2. Identify the components to be changed.
3. Assess the impact of the proposed change.
4. Review and evaluate it by the authority named in the configuration management plan (in practice a Change Control Board).

5 Decide: if not approved, the change is rejected; if approved, continue.

6 Implement the change on a copy of the code in the development area.

7 Verify by module and system testing; if not accepted, rework it.

8 Notify users and update the configuration records.

9 Form a new baseline.

The output is an Engineering Change Order (ECO) and a new baseline.

4. Configuration auditing

Identification, version control and change control keep order only until an ECO is produced, so an audit confirms that the change was really made correctly.

- **Technical review:** focuses on the technical accuracy of the changed object, checking for omissions, inconsistency and side effects.
- **Configuration audit:** checks things a review usually misses. Was the change in the ECO made? Was a technical review done? Were the process and standards followed? Is the change marked in the item with date and author? Were the SCM procedures for noting, recording and reporting followed? Were all related items updated?
- **Two formal audits before a baseline is released:** the Physical Configuration Audit (PCA) checks that all identified items are present in the correct version; the Functional Configuration Audit (FCA) checks that each item does what its specification says.

5. Configuration status reporting

Also called status accounting. Every SCM activity is recorded and reported so that everyone knows the current state: what happened, who did it, when, and what else is affected. The reports must give the status and history of any CI and be detailed enough to rebuild any previous baseline.

Task	One-line meaning	Key output
Identification	Name and list the items	CI list, baseline
Version control	Store and track versions	Repository, change sets
Change control	Approve and implement changes	ECO, new baseline
Auditing	Verify the change	PCA and FCA reports
Reporting	Tell everyone what changed	Status reports

Q13 Explain Risk and its types. Explain the setting up or generating of the RMMM plan. 7 Marks

Reference book: Sec. 5.1 to 5.5

Risk is the possibility of suffering loss: poorer quality, higher cost, delay or failure of the project.

Types of risk: schedule or time-related risks, budget or financial risks, operational or procedural risks, technical, functional or performance risks, and unavoidable risks such as a change in government policy, obsolescence caused by a rival's new technology, or loss of contracts at the customer's end.

Generating the RMMM plan

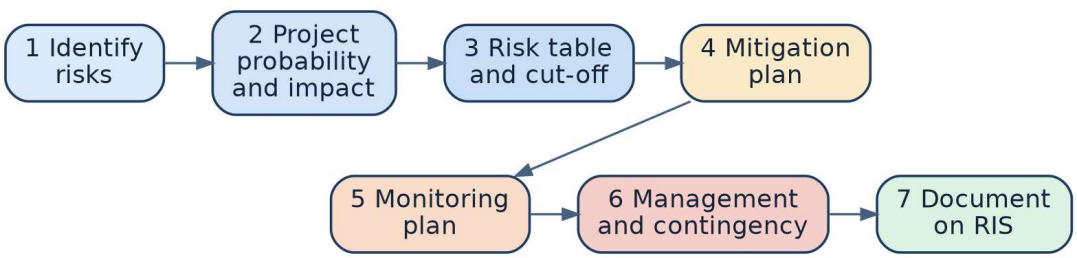


Fig. 1: Steps in generating the RMMM plan

1. **Identify the risks** using risk-item checklists and mark the category of each.
2. **Project the risk:** set a scale for likelihood (for example 1 to 10), describe the consequences, estimate the impact on the project and the product, and note how accurate the estimate is. The impact depends on three things: its **nature** (what problems it causes), its **scope** (how far it spreads and how severe it is) and its **timing** (when and for how long it is felt). Risk exposure is **RE = P × C**.
3. **Build the risk table** with the risk, category, probability, impact (1 catastrophic to 4 negligible) and a pointer to the RMMM entry. Sort it by probability and impact and draw the cut-off line.
4. **Mitigation plan:** actions that avoid the risk before it occurs (for staff turnover: find the causes, keep backup staff, set documentation standards).
5. **Monitoring plan:** the factors to watch and checks that mitigation is working; collect data for future analysis.
6. **Management and contingency plan:** what to do if the risk occurs (use backups, refocus resources, bring newcomers up to speed).
7. **Document** every risk on a Risk Information Sheet (RIS) and keep updating it as the project proceeds.

WORKED EXAMPLE

Risk = staff turnover, P = 0.70, cost if it happens = ₹2,00,000. RE = 0.70 × 2,00,000 = **₹1,40,000**. This is high, so the risk stays above the cut-off line and gets a full RMMM entry.

REMEMBER

The seven steps: **'I Predict Trouble; Mitigate, Monitor, Manage, Document'**. Impact depends on **N-S-T** (Nature, Scope, Timing).

Q15 Compare FTR and Walkthrough. — 7 Marks

Reference book: Sec. 5.15 to 5.17

A **Formal Technical Review (FTR)** is a planned, controlled review meeting of a work product, held to give assurance of software quality. Walkthroughs and inspections are kinds of FTR. A **walkthrough** is also a review meeting, but it is less formal than an inspection.

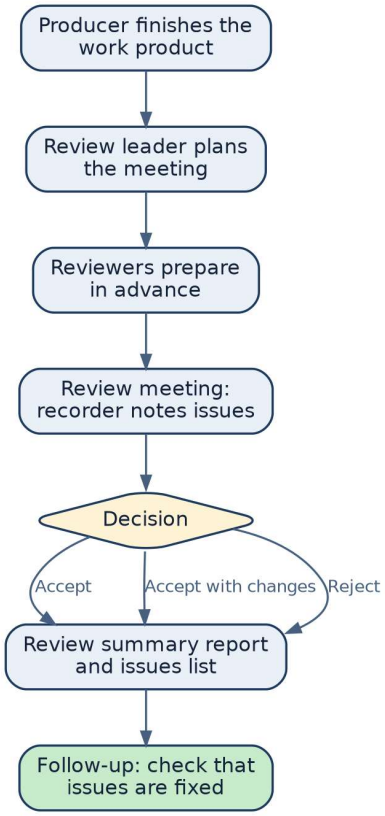


Fig. 1: Flow of a formal technical review

Parameter	FTR	Walkthrough
Concept	A process performed to assure software quality	A review meeting that is less formal than an inspection
Started by	Planned and led by a review leader	Started by the author (producer) of the product
Process	Reviewers prepare in advance and look for defects in the work product	The producer describes the product and asks the participants for comments
Formality	Formal, with the roles of leader, producer, reviewers and recorder	Informal peer review without management involvement
Size and time	3 to 5 people, at most 2 hours of preparation each, meeting under 2 hours	No fixed rule; kept short
Output	A review issues list and a one-page review summary report (what was reviewed, who reviewed it, the findings)	Suggestions noted from the participants; no formal summary report
Aim	Find defects early, check standards, make the project more manageable, train junior developers	Understand and explain the document, check the validity of the proposed system, collect suggestions

Roles in an FTR

Role	Job
Review leader	Checks that the product is ready, schedules the meeting and distributes the material
Producer	The author; explains the product to the reviewers
Reviewers	Prepare in advance and raise issues
Recorder	Writes down every issue raised

Review guidelines: keep the group to 3 to 5 people, limit advance preparation to 2 hours each, keep the meeting short, review one small work product instead of the whole design, summarise the issues at the end

of the meeting, and give the follow-up to the review leader so that nothing is forgotten.

REMEMBER

FTR = **Formal Team Review** (planned, led by a leader, produces a report). Walkthrough = **Walk with the author** (author-led and relaxed).